Practical: 06

Aim: Use of Fourier transform for filtering the image.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read image in grayscale
image = cv2.imread("img6.jpg", cv2.IMREAD_GRAYSCALE)

# Compute Fourier Transform
f = np.fft.fft2(image)           # FT unshifted
fshift = np.fft.fftshift(f)      # shift zero freq to center
magnitude_spectrum = 20 * np.log(np.abs(fshift)+1)

# ================= Low-Pass Filter =================
rows, cols = image.shape
crow, ccol = rows//2, cols//2
radius = 30  # radius of low-pass filter

# Ideal Low-Pass Filter Mask
lpf_mask = np.zeros((rows, cols), np.uint8)
cv2.circle(lpf_mask, (ccol, crow), radius, 1, -1)

# Apply LPF
fshift_lpf = fshift * lpf_mask
f_ishift_lpf = np.fft.ifftshift(fshift_lpf)
img_lpf = np.fft.ifft2(f_ishift_lpf)
img_lpf = np.abs(img_lpf)

# Magnitude spectrum of LPF
lpf_spectrum = 20 * np.log(np.abs(fshift_lpf)+1)

# Gaussian Low-Pass Filter
x = np.linspace(-ccol, ccol, cols)
y = np.linspace(-crow, crow, rows)
X, Y = np.meshgrid(x, y)
D = np.sqrt(X**2 + Y**2)
sigma = 30
gaussian_lpf_mask = np.exp(-(D**2)/(2*(sigma**2)))

fshift_gauss_lpf = fshift * gaussian_lpf_mask
f_ishift_gauss_lpf = np.fft.ifftshift(fshift_gauss_lpf)
img_gauss_lpf = np.abs(np.fft.ifft2(f_ishift_gauss_lpf))
gauss_lpf_spectrum = 20 * np.log(np.abs(fshift_gauss_lpf)+1)

# ================= High-Pass Filter =================
# Ideal High-Pass Filter
```

```
hpf_mask = 1 - lpf_mask
fshift_hpf = fshift * hpf_mask
f_ishift_hpf = np.fft.ifftshift(fshift_hpf)
img_hpf = np.abs(np.fft.ifft2(f_ishift_hpf))
hpf_spectrum = 20 * np.log(np.abs(fshift_hpf)+1)

# Gaussian High-Pass Filter
gaussian_hpf_mask = 1 - gaussian_lpf_mask
fshift_gauss_hpf = fshift * gaussian_hpf_mask
f_ishift_gauss_hpf = np.fft.ifftshift(fshift_gauss_hpf)
img_gauss_hpf = np.abs(np.fft.ifft2(f_ishift_gauss_hpf))
gauss_hpf_spectrum = 20 * np.log(np.abs(fshift_gauss_hpf)+1)

# ================= Display using Matplotlib =================

plt.figure(figsize=(12, 12))

plt.title("Fourier transform for filtering the image 23_12")

plt.subplot(3, 3, 1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 2)
plt.title("FFT Magnitude Spectrum")
plt.imshow(magnitude_spectrum, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 3)
plt.title("Low-Pass Filter Mask")
plt.imshow(lpf_mask, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 4)
plt.title("Low-Pass Filtered Image")
plt.imshow(img_lpf, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 5)
plt.title("LPF Spectrum")
plt.imshow(lpf_spectrum, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 6)
plt.title("Gaussian LPF Image")
plt.imshow(img_gauss_lpf, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 7)
plt.title("Gaussian LPF Spectrum")
```

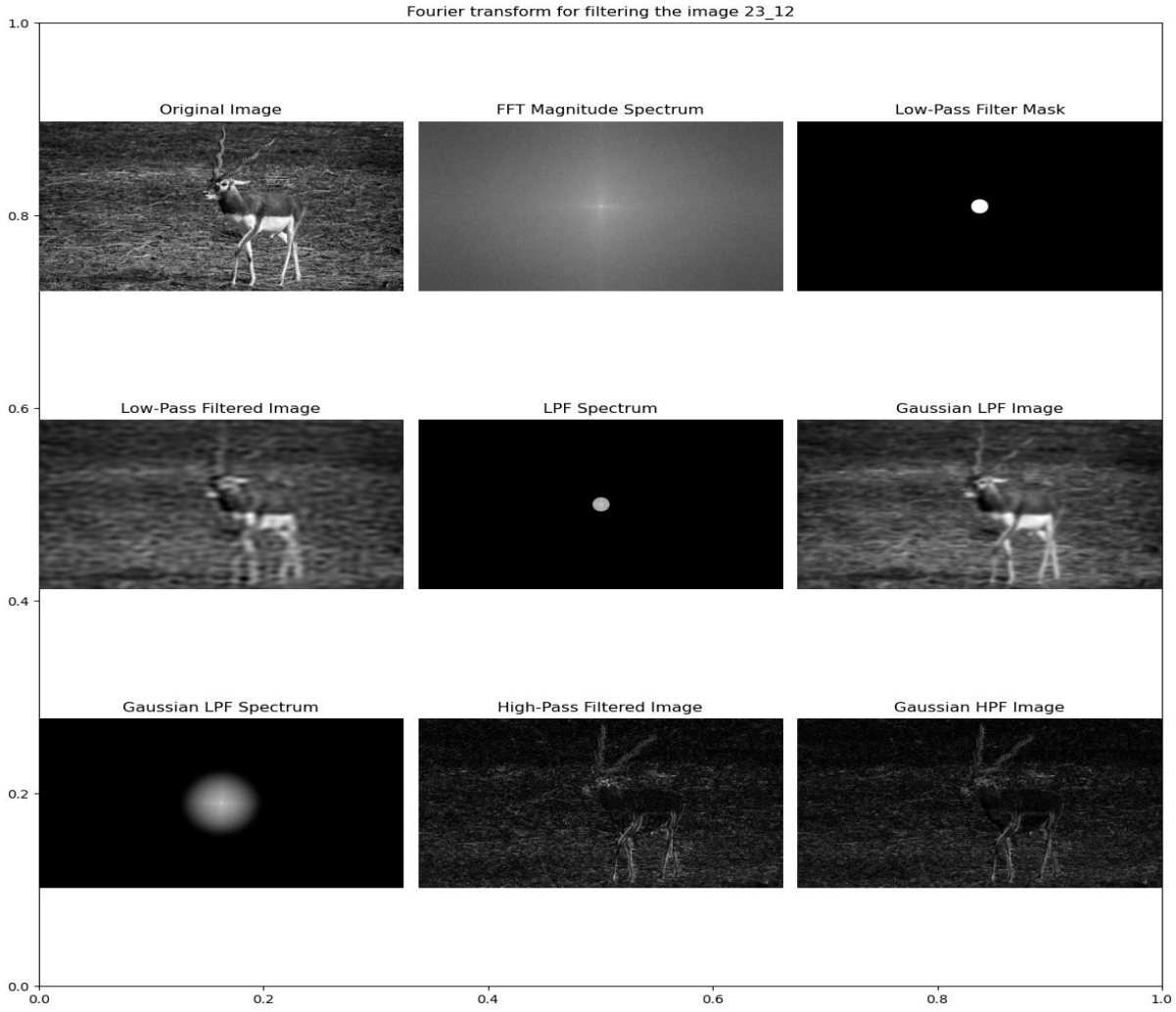
```
plt.imshow(gauss_lpf_spectrum, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 8)
plt.title("High-Pass Filtered Image")
plt.imshow(img_hpf, cmap='gray')
plt.axis('off')

plt.subplot(3, 3, 9)
plt.title("Gaussian HPF Image")
plt.imshow(img_gauss_hpf, cmap='gray')
plt.axis('off')

plt.tight_layout()
plt.show()
```

Output:



Practical: 07

Aim: Utilization of SHIFT and HOG feature for image analysis.

Code:-

SHIFT OPERATION:

```
import cv2
# Loading the image
img = cv2.imread('sample_2.jpg')
if img is None:
   print("Error: Image not loaded. Check the file path and filename.")
   exit()
cv2.imshow("Original",img)
 # Converting image to grayscale
gray= cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)
# Applying SIFT detector
sift = cv2.SIFT_create()
kp = sift.detect(gray, None)
# Marking the keypoint on the image using circles
img=cv2.drawKeypoints(gray ,
                      kp ,
                      img ,
                      flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
cv2.imshow('image-with-keypoints', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:-

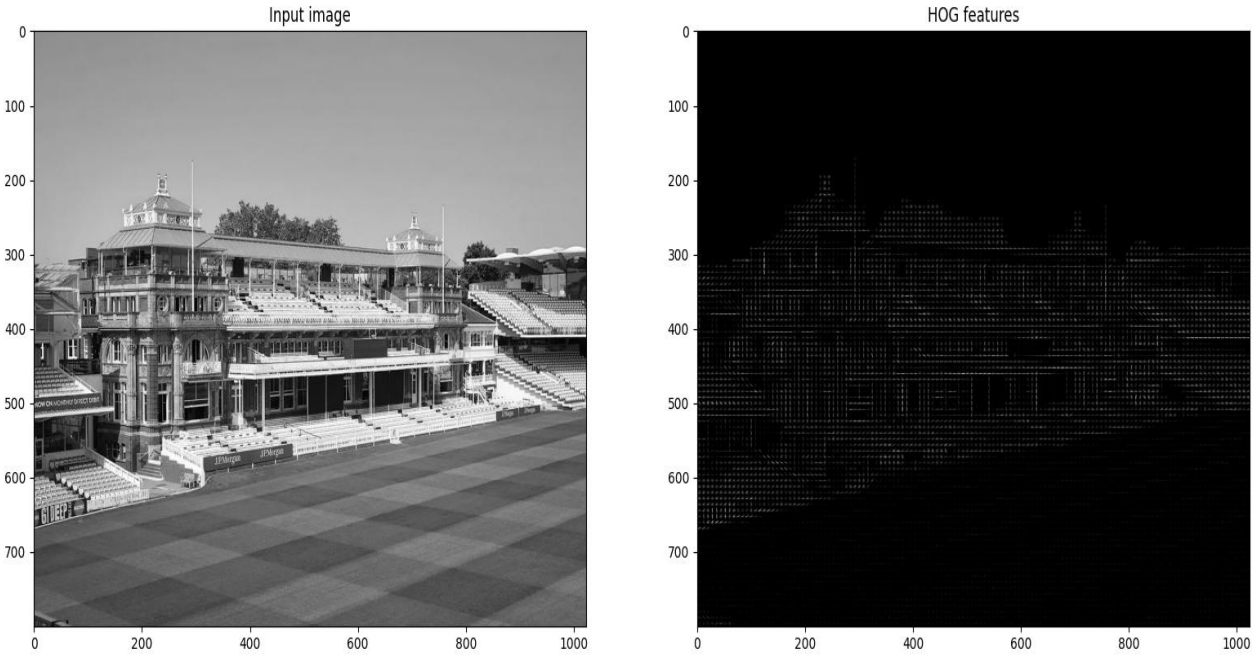
HOG features:

Code:-

```
# -*- coding: utf-8 -*-
from skimage import color
from skimage.feature import hog
from skimage import data, exposure, io
import matplotlib.pyplot as plt
import cv2
# Loading an example image
image = cv2.imread("prac_8_final.jpg")
image_gray = color.rgb2gray(image) # Converting image to grayscale
# Extract HOG features
features, hog_image = hog(image_gray, orientations=9, pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2), visualize=True)

plt.figure(figsize=(8, 4))
plt.subplot(1, 2, 1)
plt.imshow(image_gray, cmap='gray')
plt.title('Input image')
plt.subplot(1, 2, 2)
plt.imshow(hog_image, cmap='gray')
plt.title('HOG features')
plt.show()
```

Output:-



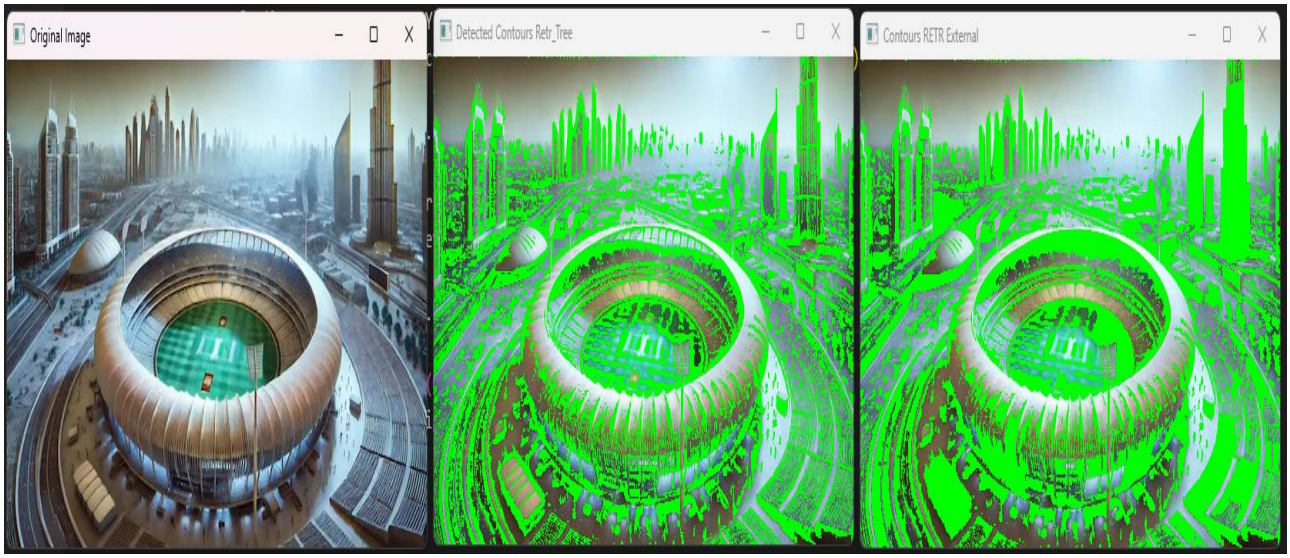
Practical: 08

Aim: Performing image segmentation: Implement Active Contour Detection.

Code:-

```
#imports
import cv2
from matplotlib import pyplot as plt
import numpy as np
#loading img
img = cv2.imread("prac_8_.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.imshow("Original Image",img)
#conversion
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
thresh = cv2.adaptiveThreshold(gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV, 21, 5)
#Func to find contours
contours, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
detected_contours = img.copy()
cv2.drawContours(detected_contours, contours, -1, (0, 255, 0), -1)
cv2.imshow("Detected Contours Retr_Tree",detected_contours)
#RETR EXTERNAL
contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
highlight = img.copy()
cv2.drawContours(highlight, contours, -1, (0, 255, 0), -1)
cv2.imshow("Contours RETR External ",highlight)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output:-



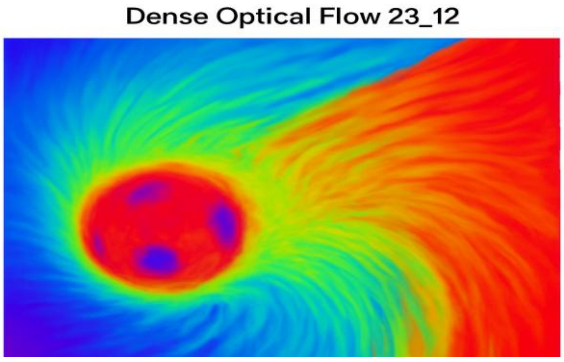
Practical: 09

Aim: Demonstrate the use of optical flow in any image processing application.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
# 1. Load image
frame1 = cv2.imread("frame1.jpg")
if frame1 is None or frame2 is None:
   raise FileNotFoundError("Frames not found. Check paths!")
# Convert to grayscale
prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
next = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)
# 2. Compute Optical Flow (Farneback)
flow = cv2.calcOpticalFlowFarneback(prvs, next, None,
                                    0.5, 3, 15, 3, 5, 1.2, 0)
# 3. Visualize optical flow
# Get flow magnitude and angle
magnitude, angle = cv2.cartToPolar(flow[...,0], flow[...,1])
# Create HSV image
hsv = np.zeros_like(frame1)
hsv[...,1] = 255
# Angle corresponds to hue, magnitude to value
hsv[...,0] = angle * 180 / np.pi / 2
hsv[...,2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)
# Convert HSV to RGB for visualization
rgb_flow = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)
# 4. Display results
fig, ax = plt.subplots(2, 1, figsize=(8, 12))  # 2 rows, 1 column
ax[0].imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
ax[0].set_title("Image 23_12")
ax[0].axis("off")
ax[1].imshow(rgb_flow)
ax[1].set_title("Dense Optical Flow 23_12")
ax[1].axis("off")
plt.tight_layout()
plt.show()
```

Output:



Practical: 10

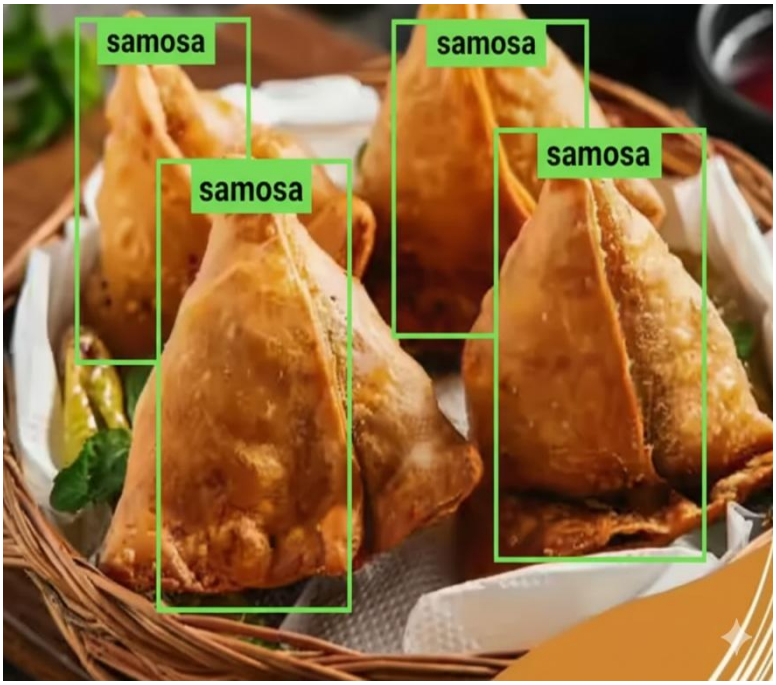
Aim: Object detection and Recognition on available online image datasets.

Code:

```
# Install required packages (run only in notebook or terminal)
!pip install ultralytics opencv-python matplotlib
import cv2
import numpy as np
import matplotlib.pyplot as plt
from ultralytics import YOLO
# 1. Load Local Image
img = cv2.imread("img10.jpg")  # replace with your local image path
if img is None:
   raise FileNotFoundError("Image not found! Please check the path.")
# Convert BGR to RGB for Matplotlib
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
# 2. Load YOLOv8 Model
model = YOLO("yolov8n.pt")  # YOLOv8 nano pretrained model
# 3. Detect Objects
results = model(img_rgb)
# Get detection image
detected_img = results[0].plot()  # returns image with bounding boxes
# 4. Display Original & Detected Images Horizontally
fig, ax = plt.subplots(2, 1, figsize=(12,12))
ax[0].imshow(img_rgb)
ax[0].set_title("Original Image 23_12")
ax[0].axis("off")

ax[1].imshow(detected_img)
ax[1].set_title("Object Detection 23_12")
ax[1].axis("off")
plt.tight_layout()
plt.show()
```

Output:-



Beyond Syllabus

Practical: 11

Aim: Object detection and Recognition on available online image datasets.

Code:

```
import cv2 import
numpy as np
# Open the input image
img = Image.open("123454210.avif")

# Translation distances
tx, ty = 100, 50  # move 100px right, 50px down

# Create a new blank image (same mode and size)
translated_img = Image.new(img.mode, img.size, (0, 0, 0))

# Paste the original image shifted
translated_img.paste(img, (tx, ty))

# Save output image
translated_img.save("translated_image.png")

print(" Image translated and saved as translated_image.png")
```

Output:-

Practical: 12

Aim: Object detection and Recognition on available online image datasets.

Code:

```
from PIL import Image # Load the image
 image_path = "input.jpg"
   img = Image.open(image_path)

# --- Rotation ---
angle = 45
  rotated_img = img.rotate(angle, expand=True)
 rotated_img.save("rotated_image.jpg")

# --- Scaling ---
scale_factor = 1.5   new_width = int(img.width * scale_factor)
new_height = int(img.height * scale_factor)
scaled_img = img.resize((new_width, new_height))
 scaled_img.save("scaled_image.jpg")

# --- Show both images ---
 rotated_img.show(title="Rotated Image")
scaled_img.show(title="Scaled Image")
print(" Rotation and scaling done! Output images saved as:")
print(" - rotated_image.jpg")
print(" - scaled_image.jpg")
```

Output:-

Practical: 13

Aim: Write a python program to perform Red and Green colour detection and tracking on Video.

Code:-

```
import cv2 import
numpy as np
From PIL import image
# Load the image (simulate video frame)
image_path = "1dab9ddd-1f71-41fa-
9e1c-422be0bb5de2.png" img =
cv2.imread(image_path)
# Convert to HSV color space
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Define HSV range
for RED color
lower_red1 =
np.array([0, 120, 70])
upper_red1 =
np.array([10, 255,
255]) lower_red2 =
np.array([170, 120,
70])
upper_red2 = np.array([180, 255, 255])

# Combine both red ranges
mask_red = cv2.inRange(hsv, lower_red1, upper_red1) + cv2.inRange(hsv,
lower_red2, upper_red2)

# Define HSV range for
GREEN color
lower_green =
np.array([36, 50, 70])
upper_green =
np.array([89, 255,
255])
mask_green = cv2.inRange(hsv, lower_green, upper_green)

# Find contours for red and green regions
```

```
contours_red, _ = cv2.findContours(mask_red, cv2.RETR_TREE,
cv2.CHAIN_APPROX_SIMPLE) contours_green, _ = cv2.findContours(mask_green,
cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

# Draw contours
on the original
image for
contour in
contours_red:
   area =
cv2.contourArea(c
ontour)    if area >
100:  # ignore
small spots        x,
y, w, h =
cv2.boundingRect(
contour)
       cv2.rectangle(img, (x, y), (x + w, y + h), (0, 0, 255), 2)
cv2.putText(img, "Red", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0, 0, 255), 2)
for contour in contours_green:
   area =
cv2.contour
Area(conto
ur)    if area
> 100:

 x, y, w, h = cv2.boundingRect(contour)

  cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
   cv2.putText(img, "Green", (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0),
2)

# Save the output
image with detection
boxes output_path =
"output_color_detecti
on.jpg"
cv2.imwrite(output_pa
th, img)

# Convert to PIL and show
```

```
Image.open(output_path).show()

print(" Red and Green color detection done. Output saved as:", output_path)
```

Output:-